Project Proposal : Agent Hospital 2.0

Philippe Schoeb
Mila
University of Montreal
philippe.schoeb@umontreal.ca

Nhung Dang
Rali
University of Montreal
thi.hong.nhung.dang@umontreal.ca

Abstract

The use of Large Language Models (LLMs) in medical fields can improve health system in many ways. For instance, artificial doctor agents can be of help to actual doctors to accelerate and improve their daily tasks. A recent and highly effective approach to construct state of the art agents in this field was developed and shared earlier this year. It is called **Simulacrum-based Evolutionary Agent Learning (SEAL)** and it was developed in the context of **Agent Hospital** (Li et al., 2025). For this project, we aim to implement a simplified version of their SEAL model to discover if it remains efficient even when decreasing its complexity. We also aim to explore different modifications that could be applied to further understand how it works.

1 Introduction

The use of LLMs has become predominant in many fields thanks to the rapid improvements of Deep Learning. Nowadays, several different methods for constructing capable LLM agents are used. For this project, we will be focusing on **Agent Hospital** (Li et al., 2025) which is a state of the art AI technique for building virtual medical agents. The essence of this technique is to have an LLM agent with basic medical knowledge at the start and to have it learn about medicine through its numerous interactions with patients in a simulation. Such efficient agents can facilitate the work of doctors resulting in more patients consulted in addition to higher quality diagnoses and treatment proposals. Agent Hospital shows promising results as it outperforms existing methods on the **MedQA** [1] dataset (Jin et al., 2020) with GPT-4o as the base model and it showcases many advantages such as not needing annotated training data. Our goal is to implement a simpler version of their SEAL model to not only show how their model reacts to a complexity reduction but also to explore different model modifications in order to get a grasp on it. Our version of Agent Hospital will not support the simulation of nurses' tasks and will focus exclusively on the treatment of patients by doctors. This is important because it would help optimize and understand future implementations of such a model. Indeed, if a decrease in complexity saves a lot of money and still produces a good model, then that would be good to know. Furthermore, model exploration is important for the same reasons. In the end, we wish to find new experimental results to contribute to the analysis of Agent Hospital.

[1] **MedQA** is available at https://github.com/jind11/MedQA

2 Task and dataset

In Agent Hospital, a medical agent, also known as a doctor agent in our work, will have two main types of tasks. The first type involves **virtual tasks**, meaning all tasks performed within the simulation. These include the diagnosis task, which consists of identifying the correct disease, and the treatment plan recommendation task, which involves recommending an appropriate treatment plan for artificial patients. We plan on implementing the former task only and we will see about the latter one, depending on how the project advances. All tasks in this virtual environment will be based on the knowledge the agent acquired through initialization and through its interactions with previous patients. To initialize a doctor agent with basic medical knowledge, we plan on using the textbooks from the MedQA dataset. Once the disease has been identified and the treatment has been given by the agent doctor, the result of this prediction can be seen as the patient's feedback and will contribute to the evolution of the agent doctor.

The second type of task is **real-life tasks**, which

focuses on aligning the agent's knowledge with real-world medical expertise beyond the simulation. The experiences accumulated in the virtual world can be leveraged to help solve real-world medical problems. For this, we will use the MedQA dataset, which is derived from professional medical board exams, to evaluate the doctor agents.

Moreover, we plan to use different LLMs as the base of our medical agents to see how that choice affects Agent Hospital. Namely, we plan to use **GPT** and **Aeiva** [2]. For the generation of patient agents, we cannot use the same health encyclopedia used for Agent Hospital since it is in Chinese. We could use the **A.D.A.M. Medical Encyclopedia** [3] depending on its availability. However, we would need to identify the most common diseases beforehand as they will be more important for the medical agents. The method for finding these most common diseases still needs to be determined as the source used by Agent Hospital cannot be used again.

3 Methods

We plan to build a simplified version of the SEAL model implemented in Agent Hospital. First, instead of having a complex hospital simulation cycle, we will implement a much simpler process which includes the generation of a patient and doctor consultation (diagnosis). Second, we would also like to simulate with a single doctor handling all different medical departments instead of having specialized doctors from each department. Nevertheless, for our first implementation, we will only be targeting diseases from a single medical department: **cardiology**. Some other exploration ideas we have are :

- **Impact of pre-training on doctor agent evolution**: Having some doctor agents start without any medical expertise and some with pre-training on the MedQA textbooks to see how it affects their evolution.
- **Impact of noise on model reinforcement**: Verifying the impact of noise by mixing patients' symptoms to assess the model's ability to detect the most relevant signs and provide appropriate suggestions.

[2] **Aeiva** https://chatsci.github.io/Aeiva/

[3] **A.D.A.M. Medical Encyclopedia** https://medlineplus.gov/encyclopedia.html

4 Related work

4.1 Patient agent generation

As mentionned in the previous sections, we plan to use the A.D.A.M. Medical Encyclopedia[3] knowledge base to generate patient agents using a LLMs such as GPT and Aeiva. If GPT is used for patient data generation, Aeiva will be used to train the agent doctor, and vice versa. The selected model will generate basic patient information, such as name, age, gender, and medical history, based on a randomly selected disease. Additionally, we leverage the model to create a medical history by incorporating certain symptoms from others diseases to introduce some noise.

4.2 Doctor agent response generation

By providing a patient agent to the doctor agent, along with its basic information, medical history, and symptoms without noise generated by the LLM, the objective of the doctor agent is to make a wise decision based on the medical examination, diagnosis, and prescription of medications. This work aims to identify the correct disease among the relevant symptoms, propose an appropriate treatment plan, and finally generate a comprehensive response for the patient.

4.3 Doctor agent response evolution

The evolution of a doctor agent consists of two main aspects. The first involves the automatic updating of knowledge to correct errors when a prediction about a patient agent's disease has been incorrect, and the second aims to automatically reinforce its knowledge when the prediction of a patient's disease has been correct.

If an incorrect prediction of a patient's disease has been made, the doctor agent will switch to its **auto-reflection mode**. This mode aims to compare its decision with the ground-truth decisions and adjust its reasoning to archive a correct decision, i.e., correctly treating the patient by identifying the right disease. Each time the doctor agent successfully corrects a prediction error, a rule will be added to its experience base to prevent making the same mistake in the future. This rule could be constructed in the same way as **turning-free rule accumulation** (Yang et al., 2023).

If the doctor agent has correctly treated a patient agent, the case will be added to the medical base of that agent. In order to reinforce the model, two options can be used to introduce noise into the current

case. A random symptom from another disease can be incorporated into the case, or a symptom from the current case can be randomly removed. If the agent still correctly identifies the original disease, the case remains in the medical base. Otherwise, the self-reflection mode will be restarted to determine a better rule for inclusion in the experience base.

As described, each doctor agent will have its own medical base and experience base for its evolution. These bases will expand as the number of treated patients and auto-generated cases grows. With these two modules, doctor agents continuously improve their capabilities and reinforce themselves as the number of treated patient agents and cases increases.

5 Baselines and evaluation

Based on what we explained in the Related work section, we are mainly concerned with two aspects of quality. The first is how the medical data of a generated patient adheres to our medical knowledge base. The second is which reference base will be used to evaluate the accuracy of the doctor agent in both the virtual environment and real life.

5.1 Quality control in patient agent generation

Given a generated patient based on our chosen medical knowledge base, the A.D.A.M. Medical Encyclopedia, we plan to experiment with frameworks such as **RAGAS**[4] (Es et al., 2023), **ARES**[5] (Saad-Falcon et al., 2024), et **G-Eval**[6] (Liu et al., 2023) to assess the quality of content generated by Natural Language Generation (NLG) system. The goal of this step is to identify the most effective framework for evaluating generated patient.

5.2 Evaluation of doctor agent prediction and evolution

There are two contexts in which the doctor agents will be evaluated. For both, we can compare our results with what is provided in Agent Hospital. **In virtual environment**: To conduct this evaluation, based on the data available from the A.D.A.M. Medical Encyclopedia, each generated patient agent will be assigned a ground-truth disease along with a set of associated symptoms. Once the doctor agent provides a diagnosis, we can use the text similarity techniques such as **TF-IDF** or **cosine similarity** to determine the similarity score between the ground-truth disease and the doctor agent's predicted disease. If we receive a high score, we consider the diagnosis correct; otherwise, it is incorrect.

In this work, we will first focus on disease prediction. If the diagnosis is incorrect, the entire treatment suggestion will be ignored. Otherwise, we will apply the same technique to evaluate the validity of treatment plan. However, depending on the available time, this evaluation may or may not be implemented.

In real-life: We can directly use the questions from the MedQA dataset. These are multiple choice questions concerning every aspect of medecine. A simple baseline would be to evaluate a LLM that has not been in the simulation.

[4]**RAGAS** https://github.com/explodinggradients/ragas

[5]**ARES** https://github.com/stanford-futuredata/ARES

[6]**G-Eval** https://github.com/nlpyang/geval

References

Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2023. Ragas: Automated evaluation of retrieval augmented generation. *Preprint*, arXiv:2309.15217.

Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. 2020. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Preprint*, arXiv:2009.13081.

Junkai Li, Yunghwei Lai, Weitao Li, Jingyi Ren, Meng Zhang, Xinhui Kang, Siyu Wang, Peng Li, Ya-Qin Zhang, Weizhi Ma, and Yang Liu. 2025. Agent hospital: A simulacrum of hospital with evolvable medical agents. *Preprint*, arXiv:2405.02957.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-eval: Nlg evaluation using gpt-4 with better human alignment. *Preprint*, arXiv:2303.16634.

Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. Ares: An automated evaluation framework for retrieval-augmented generation systems. *Preprint*, arXiv:2311.09476.

Zeyuan Yang, Peng Li, and Yang Liu. 2023. Failures pave the way: Enhancing large language models through tuning-free rule accumulation. *Preprint*, arXiv:2310.15746.